# AMERICAN INTERNATIONAL UNIVERSITY BANGLADESH (AIUB)

*FACULTY OF SCIENCE & TECHNOLOGY*

Course Title

## INTRODUCTION TO DATABASE (CSC2108)

**Semester: Fall 2024-2025**

**Section: AA**

# Tech Shop Management System

**Supervised By**

DR. RAZUAN KARIM

**Submitted By**

**Group no: 4**

| Name | ID |
|---|---|
| Md. Mahinul Islam | 23-55084-3 |
| Shanjida Ahmed Shema | 23-53378-3 |
| Sabekon Naher | 23-53361-3 |
| Adib Afsar Khan | 23-55721-3 |

# TABLE OF CONTENTS


| TOPICS | | Page no. |
|---|---|---|
| Title Page | | 1 |
| Table of Content | | 2 |
| 1. | Introduction | 3 |
| 2. | Case Study | 4 |
| 3. | ER Diagram | 5 |
| 4. | Normalization | 6-10 |
| 5. | Finalization | 11 |
| 6. | Table Creation | 12-21 |
| 7. | Data Insertion | 22-38 |
| 8. | Query Test | 39-51 |
| 9. | DB connection | 52-57 |

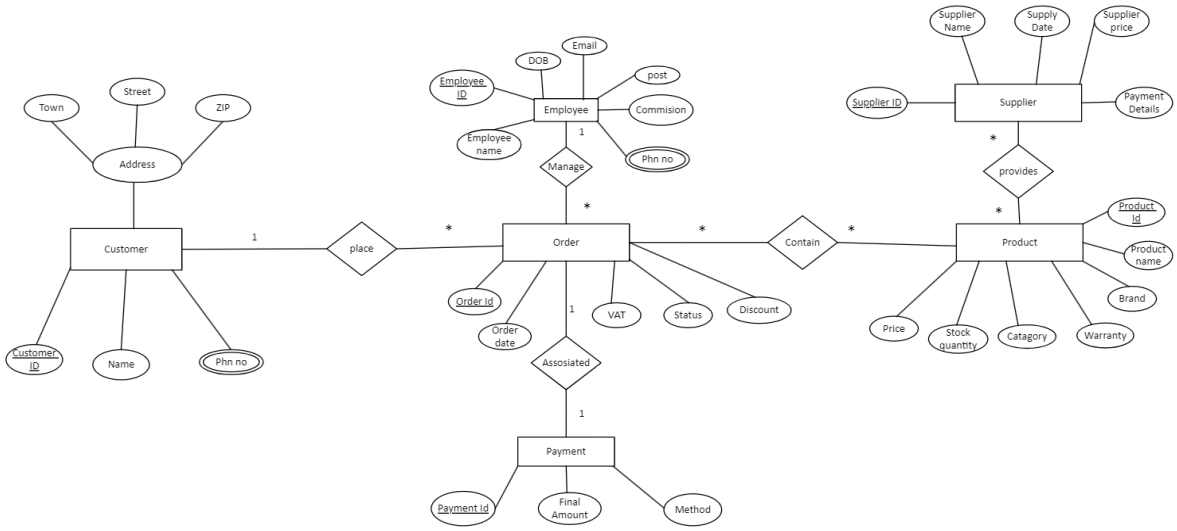
# Introduction

A Tech Shop Management System is a software tool designed to help manage the operations of a tech shop that sells electronic products like computers, mobile phones, and accessories. It makes it easier for shop owners and employees to handle tasks such as processing sales, managing customer details, and keeping financial records. By using this system, a tech shop can run more smoothly, reducing errors and saving time, making it easier to offer good service to customers. The system typically includes features like stock management, billing, and reporting, helping the shop stay organized and efficient. The project is developed using Oracle 10g for database management and Java for application development.

# Case Study

In a tech shop management system, a **customer** may place many orders. One order is associated with exactly one customer. A customer is identified by their customer_ID. The system also stores customer name, phone number and address. A customer address is composed of street, town, and zip code. A **product** is identified by a unique Product ID. The system stores product name, category, brand, price, stock quantity, and warranty period.The category specifies the type of product (e.g., electronics, accessories, appliances) and plays a key role in maintaining an organized inventory. When a customer places an **order**, an Order ID is generated. Each order includes the customer ID (phone/email), order date, and payment status. An order contains a list of products, where for each product, the product ID, name, brand, price, quantity, and total price are stored. The system also calculates VAT, discounts, total amount, and the final amount to be paid for the order.**Payments** are identified by a unique Payment ID. A payment is associated with exactly one order and includes details such as the final amount, and payment method. The system also manages **suppliers**. A supplier is identified by a unique Supplier ID. The database stores supplier name, supply date, and supply payment details. A supplier provides multiple products to the shop. **Employees**, including admins, are part of the system. An employee is identified by a unique Employee ID. The system stores employee name, phone number, email ID, date of birth, post, and commission. Employees manage customer orders, update inventory, and oversee payment processes.

# E-R Diagram


Town
Street
ZIP
Address
Customer
Customer ID
Name
Phn no
1
place
*
Employee ID
DOB
Email
post
Employee
Commision
Employee name
1
Phn no
Manage
*
Order
Order Id
Order date
1
VAT
Status
Discount
Assosiated
1
Payment
Payment Id
Final Amount
Method
*
Contain
*
Supplier Name
Supply Date
Supplier price
Supplier ID
Supplier
Payment Details
*
provides
*
Product
Product Id
Product name
Brand
Price
Stock quantity
Catagory
Warranty

# Normalization

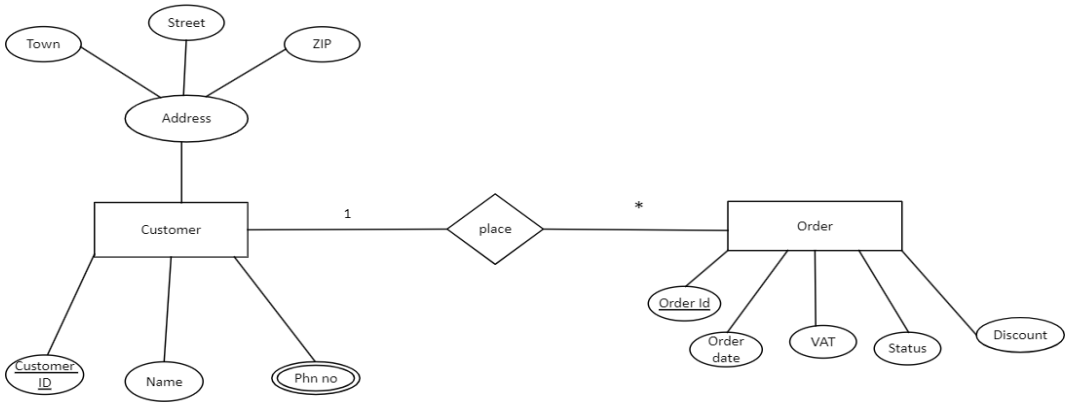



**Fig 1: Customer - Place – Order**

## Relation: One to Many

**UNF:**

1. Customer_ID, Customer_Name, Phn_No, Town,Street,Zip, Order_ID, Order_Date, Status, VAT,Discount

**1NF:**

Multivalued Attributes: Phn_No

1. Customer_ID, Customer_Name, Phn_No, Town,Street,Zip ,Order_ID, Order_Date, Status, VAT,Discount.

**2NF:**

1. Customer_ID, Customer_Name, Town,Street,Zip
2. Order_ID, Order_Date, Status, VAT, Discount
3. Customer_ID , Phn_No
4. Customer_ID , Order_ID

**3NF:**

1. Customer_ID, Customer_Name
2. Town,Street,Zip
3. Order_ID , Order_Date, Status, VAT,Discount
4. Customer_ID , Phn_No
5. Customer_ID , Order_ID

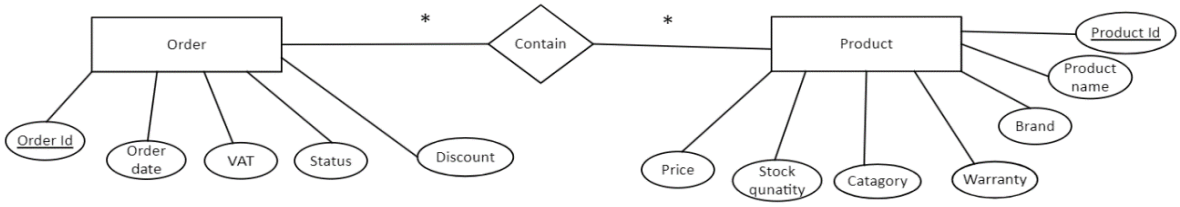



**Fig 2: Order – Contain – Product**

## Relation: Many to Many

**UNF:**

1. Order_ID, Order_Date, Status, VAT, Discount ,Product_ID, Product_Name, Price, Category, Stock Quantity,warranty,Brand

**1NF:**

No multivalued attributes

1. Order_ID, Order_Date, Status, VAT, Discount, Product_ID, Product_Name, Price,Category,Stock Quantity,warranty,Brand

**2NF:**

1. Order_ID, Order_Date, Status, VAT, Discount
2. Product_ID, Product_Name, Price, Category, Stock Quantity,warranty,Brand, Order_ID
3. Order_ID , Product_ID

**3NF:**

1. Order_ID, Order_Date, Status, VAT, Discount
2. Product_ID, Product_Name, Price
3. Category, Stock Quantity, warranty, Brand
4. Order_ID , Product_ID

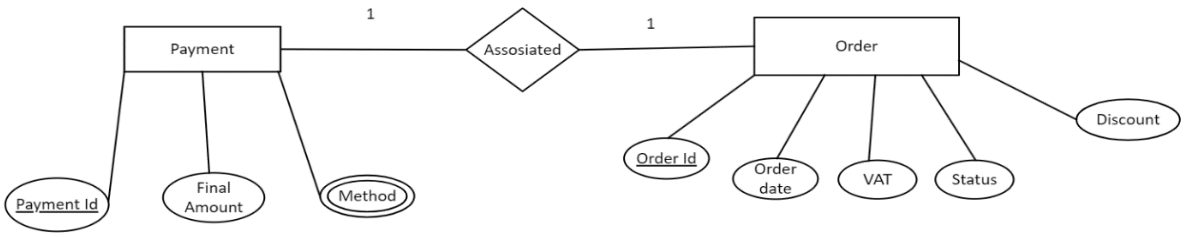



**Fig 3: Payment – Assosiated – Order**

## Relation: One to One

**UNF:**

1. Order_ID, Order_Date, Status, VAT, Discount , Payment_ID, Payment_Method, Final_Amount

**1NF:**

Multivalued attributes: Payment_Method

1. Order_ID, Order_Date, Status, VAT, Discount , Payment_ID, Payment_Method, Final_Amount

**2NF:**

1. Order_ID, Order_Date, Status, VAT, Discount
2. Payment_ID, Final_Amount
3. Order_ID , Payment_ID
4. Payment_ID , Payment_Method

**3NF:**

As there is no transitive dependency in 2NF , that's why no need to add 3NF

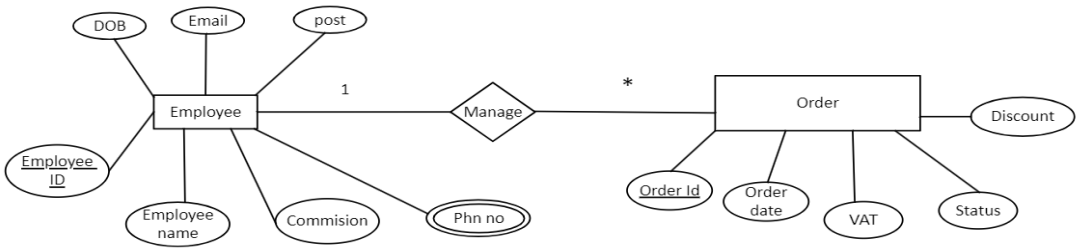



**Fig 4: Employee – Manage – Order**

## Relation: One to Many

**UNF:**

1. Order_ID, Order_Date, Status, VAT, Discount, Employee_ID, Employee_Name, Post, Commission, Phn_No,DOB,Email

**1NF:**

Multivalued Attributes: Phn_No

1. Order_ID, Order_Date, Status, VAT, Discount, Employee_ID, Employee_Name, Post, Commission, Phn_No,DOB,Email

**2NF:**

1. Order_ID, Order_Date, Status, VAT, Discount
2. Employee_ID, Employee_Name, Post, Comm,DOB,Email
3. Order_ID , Employee_ID
4. Employee_ID , Phn_No

**3NF:**

1. Order_ID, Order_Date, Status, VAT, Discount
2. Employee_ID, Employee_Name,DOB
3. Post, Comm,Email
4. Order_ID, Employee_ID
5. Employee_ID , Phn_No

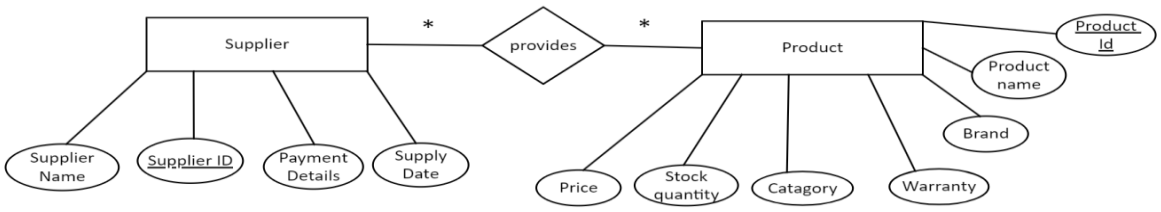



**Fig 5: Supplier - Provides – Product**

## Relation: Many to Many

**UNF:**

1. Supplier_ID, Supplier_Name, Payment_Details, Supply_Date, Product_ID, Product_Name, Price, Warranty, Stock_Quantity,Brand,Catagory

**1NF:**

No Multivalued Attributes

2. Supplier_ID, Supplier_Name, Payment_Details, Supply_Date, Product_ID, Product_Name, Price, Warranty, Stock_Quantity,Brand,Catagory

**2NF:**

1. Supplier_ID, Supplier_Name, Payment_Details, Supply_Date
2. Product_ID, Product_Name, Price, Warranty, Stock_Quantity,Brand,Catagory
3. Supplier_ID, Product_ID.

**3NF:**

1. Supplier_ID, Supplier_Name, Payment_Details, Supply_Date
2. Product_ID, Product_Name, Price
3. Warranty, Stock_Quantity,Brand,Catagory
4. Supplier_ID, Product_ID

# Finalization

1. Customer_ID, Customer_Name
2. Town,Street,Zip
3. Order_ID , Order_Date, Status, VAT,Discount
4. Customer_ID , Phn_No
5. Customer_ID , Order_ID
6. ~~Order_ID, Order_Date, Status, VAT, Discount~~
7. Product_ID, Product_Name, Price
8. Category, Stock Quantity, warranty, Brand
9. Order_ID , Product_ID
10. ~~Order_ID, Order_Date, Status, VAT, Discount~~
11. Payment_ID, Final_Amount
12. Order_ID , Payment_ID
13. Payment_ID , Payment_Method
14. ~~Order_ID, Order_Date, Status, VAT, Discount~~
15. Employee_ID, Employee_Name,DOB
16. Post, Comm,Email
17. Order_ID, Employee_ID
18. Employee_ID , Phn_No
19. Supplier_ID, Supplier_Name, Payment_Details, Supply_Date
20. ~~Product_ID, Product_Name, Price~~
21. ~~Category, Stock_Quantity,Brand,Warranty~~
22. Supplier_ID, Product_ID

# Table Creation (DDL Operations)

| | |
|---|---|
| StudentID1: 23-55084-3<br>Name: Md. Mahinul Islam | StudentID3: 23-53361-3<br>Name: Sabekon Naher |
| StudentID2: 23-53378-3<br>Name: Shanjida Ahmed Shema | StudentID4: 23-55721-3<br>Name: Adib Afsar Khan |
| **CO4**: Creating DML, DDL using Oracle and connection with ODBC/JDBC for existing JAVA application | |
| **PO-e-2:** Use modern engineering and IT tools for prediction and modeling of complex computer science and engineering problem | Marks |

##

```
create user project identified by techshop;
```

Results Explain Describe Saved SQL History

```
User created.
```

```
grant create table,create view,create sequence to project;
```

Results Explain Describe Saved SQL History

```
Statement processed.
```

1.Customer Table:

User: PROJECT

Home > SQL > SQL Commands

Autocommit Display 10

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(189)
);
DESC Customer;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **CUSTOMER**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| CUSTOMER | CUSTOMERID | Number | - | - | 0 | 1 | - | - | - |
| | CUSTOMERNAME | Varchar2 | 189 | - | - | - | ✓ | - | - |

1 - 2

2.Address Table:

```
CREATE TABLE Address (
    CustomerID NUMBER PRIMARY KEY,
    Town VARCHAR(100),
    Street VARCHAR(100),
    Zip VARCHAR(20),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
DESC Address;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **ADDRESS**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| ADDRESS | CUSTOMERID | Number | - | - | 0 | 1 | - | - | - |
| | TOWN | Varchar2 | 100 | - | - | - | ✓ | - | - |
| | STREET | Varchar2 | 100 | - | - | - | ✓ | - | - |
| | ZIP | Varchar2 | 20 | - | - | - | ✓ | - | - |

1 - 4

3.Orders Table:

```
CREATE TABLE Orders (
    OrderID NUMBER PRIMARY KEY,
    OrderDate DATE,
    Status VARCHAR(50),
    VAT NUMBER(5, 2),
    Discount NUMBER(5, 2)
);
DESC Orders;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **ORDERS**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| ORDERS | ORDERID | Number | - | - | 0 | 1 | - | - | - |
| | ORDERDATE | Date | 7 | - | - | - | ✓ | - | - |
| | STATUS | Varchar2 | 50 | - | - | - | ✓ | - | - |
| | VAT | Number | - | 5 | 2 | - | ✓ | - | - |
| | DISCOUNT | Number | - | 5 | 2 | - | ✓ | - | - |

1 - 5

4.PhoneNumber Table:

```
CREATE TABLE PhoneNumber (
    CustomerID NUMBER,
    PhnNo VARCHAR(15),
    PRIMARY KEY (CustomerID, PhnNo),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
DESC PhoneNumber;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **PHONENUMBER**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| PHONENUMBER | CUSTOMERID | Number | - | - | 0 | 1 | - | - | - |
| | PHNNO | Varchar2 | 15 | - | - | 2 | - | - | - |

1 - 2

5.Customer_Orders Table:

```
CREATE TABLE CustomerOrder (
    CustomerID NUMBER,
    OrderID NUMBER,
    PRIMARY KEY (CustomerID, OrderID),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
);
DESC CustomerOrder;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **CUSTOMERORDER**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| CUSTOMERORDER | CUSTOMERID | Number | - | - | 0 | 1 | - | - | - |
| | ORDERID | Number | - | - | 0 | 2 | - | - | - |

1 - 2

6.Products Table:

```
CREATE TABLE Products (
    ProductID NUMBER PRIMARY KEY,
    ProductName VARCHAR(255),
    Price NUMBER(10, 2)
);
DESC Products;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **PRODUCTS**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| PRODUCTS | PRODUCTID | Number | - | - | 0 | 1 | - | - | - |
| | PRODUCTNAME | Varchar2 | 255 | - | - | - | ✓ | - | - |
| | PRICE | Number | - | 10 | 2 | - | ✓ | - | - |

1 - 3

## 7.ProductDetails Table:

```
CREATE TABLE ProductDetails (
    ProductID NUMBER PRIMARY KEY,
    Category VARCHAR(255),
    StockQuantity NUMBER,
    Warranty VARCHAR(50),
    Brand VARCHAR(255),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
DESC ProductDetails;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **PRODUCTDETAILS**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| PRODUCTDETAILS | PRODUCTID | Number | - | - | 0 | 1 | - | - | - |
| | CATEGORY | Varchar2 | 255 | - | - | - | ✓ | - | - |
| | STOCKQUANTITY | Number | - | - | 0 | - | ✓ | - | - |
| | WARRANTY | Varchar2 | 50 | - | - | - | ✓ | - | - |
| | BRAND | Varchar2 | 255 | - | - | - | ✓ | - | - |

1 - 5

## 8.OrderProducts Table:

```
CREATE TABLE OrderProducts (
    OrderID NUMBER,
    ProductID NUMBER,
    PRIMARY KEY (OrderID, ProductID),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
DESC OrderProducts;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **ORDERPRODUCTS**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| ORDERPRODUCTS | ORDERID | Number | - | - | 0 | 1 | - | - | - |
| | PRODUCTID | Number | - | - | 0 | 2 | - | - | - |

1 - 2

## 9.Payments Table:

```
CREATE TABLE PaymentMethod (
    PaymentID NUMBER,
    PaymentMethod VARCHAR(255),
    PRIMARY KEY (PaymentID, PaymentMethod),
    FOREIGN KEY (PaymentID) REFERENCES Payments(PaymentID)
);
DESC PaymentMethod;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **PAYMENTMETHOD**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| PAYMENTMETHOD | PAYMENTID | Number | - | - | 0 | 1 | - | - | - |
| | PAYMENTMETHOD | Varchar2 | 255 | - | - | 2 | - | - | - |

1 - 2

## 10.Payment_Method Table:

```
CREATE TABLE PaymentMethod (
    PaymentID NUMBER,
    PaymentMethod VARCHAR(255),
    PRIMARY KEY (PaymentID, PaymentMethod),
    FOREIGN KEY (PaymentID) REFERENCES Payments(PaymentID)
);
DESC PaymentMethod;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **PAYMENTMETHOD**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| PAYMENTMETHOD | PAYMENTID | Number | - | - | - | 1 | - | - | - |
| | PAYMENTMETHOD | Varchar2 | 255 | - | - | 2 | - | - | - |

1 - 2

11.Order_Payment Table:

```
CREATE TABLE OrderPayment (
    OrderID NUMBER,
    PaymentID NUMBER,
    PRIMARY KEY (OrderID, PaymentID),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (PaymentID) REFERENCES Payments(PaymentID)
);
DESC OrderPayment;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **ORDERPAYMENT**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| ORDERPAYMENT | ORDERID | Number | - | - | - | 1 | - | - | - |
| | PAYMENTID | Number | - | - | - | 2 | - | - | - |

1 - 2

12.Employees Table:

```
CREATE TABLE Employees (
    EmployeeID NUMBER PRIMARY KEY,
    EmployeeName VARCHAR(255) NOT NULL,
    DOB DATE NOT NULL
);
DESC Employees;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **EMPLOYEES**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| EMPLOYEES | EMPLOYEEID | Number | - | - | - | 1 | - | - | - |
| | EMPLOYEENAME | Varchar2 | 255 | - | - | - | - | - | - |
| | DOB | Date | 7 | - | - | - | - | - | - |

1 - 3

13.Employee_Details Table:

```
CREATE TABLE EmployeeDetails (
    EmployeeID NUMBER PRIMARY KEY,
    Post VARCHAR(255) NOT NULL,
    Comm NUMBER(10, 2),
    Email VARCHAR(255),
    FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID)
);
DESC EmployeeDetails;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **EMPLOYEEDETAILS**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| EMPLOYEEDETAILS | EMPLOYEEID | Number | - | - | - | 1 | - | - | - |
| | POST | Varchar2 | 255 | - | - | - | - | - | - |
| | COMM | Number | - | 10 | 2 | - | ✓ | - | - |
| | EMAIL | Varchar2 | 255 | - | - | - | ✓ | - | - |

1 - 4

14.EmployeePhones Table:

```
CREATE TABLE EmployeePhones (
    EmployeeID NUMBER,
    PhnNo VARCHAR(15),
    PRIMARY KEY (EmployeeID, PhnNo),
    FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID)
);
DESC EmployeePhones;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **EMPLOYEEPHONES**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| EMPLOYEEPHONES | EMPLOYEEID | Number | - | - | - | 1 | - | - | - |
| | PHNNO | Varchar2 | 15 | - | - | 2 | - | - | - |

1 - 2

## 15.EmployeeOrders Table:

```
CREATE TABLE EmployeeOrders (
    OrderID NUMBER,
    EmployeeID NUMBER,
    PRIMARY KEY (OrderID, EmployeeID),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID)
);
DESC EmployeeOrders;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **EMPLOYEEORDERS**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| EMPLOYEEORDERS | ORDERID | Number | - | - | - | 1 | - | - | - |
| | EMPLOYEEID | Number | - | - | - | 2 | - | - | - |

1 - 2

## 16.Suppliers:

```
CREATE TABLE Suppliers (
    SupplierID NUMBER PRIMARY KEY,
    SupplierName VARCHAR(255) NOT NULL,
    PaymentDetails VARCHAR(255),
    SupplyPrice NUMBER(10,2),
    SupplyDate DATE
);
DESC Suppliers;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **SUPPLIERS**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| SUPPLIERS | SUPPLIERID | Number | - | - | - | 1 | - | - | - |
| | SUPPLIERNAME | Varchar2 | 255 | - | - | - | - | - | - |
| | PAYMENTDETAILS | Varchar2 | 255 | - | - | - | ✓ | - | - |
| | SUPPLYPRICE | Number | - | 10 | 2 | - | ✓ | - | - |
| | SUPPLYDATE | Date | 7 | - | - | - | ✓ | - | - |

1 - 5

17.SupplierProducts:

```
CREATE TABLE SupplierProducts (
    SupplierID NUMBER,
    ProductID NUMBER,
    PRIMARY KEY (SupplierID, ProductID),
    FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
DESC SupplierProducts;
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **SUPPLIERPRODUCTS**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| SUPPLIERPRODUCTS | SUPPLIERID | Number | - | - | - | 1 | - | - | - |
| | PRODUCTID | Number | - | - | - | 2 | - | - | - |

1 - 2

# Inserted Values in the tables

1.Customer:

```
INSERT ALL
INTO Customer (CustomerID, CustomerName) VALUES (11, 'John')
INTO Customer (CustomerID, CustomerName) VALUES (21, 'Jane')
INTO Customer (CustomerID, CustomerName) VALUES (31, 'Alice')
INTO Customer (CustomerID, CustomerName) VALUES (41, 'Bob')
INTO Customer (CustomerID, CustomerName) VALUES (51, 'Charlie')
INTO Customer (CustomerID, CustomerName) VALUES (61, 'Diana')
INTO Customer (CustomerID, CustomerName) VALUES (71, 'Edward')
INTO Customer (CustomerID, CustomerName) VALUES (81, 'Fiona')
INTO Customer (CustomerID, CustomerName) VALUES (91, 'George')
INTO Customer (CustomerID, CustomerName) VALUES (100, 'Helen')
SELECT * FROM dual;
SELECT *FROM Customer;
```

Results Explain Describe Saved SQL History

| CUSTOMERID | CUSTOMERNAME |
|---|---|
| 11 | John |
| 21 | Jane |
| 31 | Alice |
| 41 | Bob |
| 51 | Charlie |
| 61 | Diana |
| 71 | Edward |
| 81 | Fiona |
| 91 | George |
| 100 | Helen |

10 rows returned in 0.00 seconds CSV Export

2.Address:

```
INSERT ALL
INTO Address (CustomerID, Town, Street, Zip) VALUES (11, 'New York', 'Avenue', '10001')
INTO Address (CustomerID, Town, Street, Zip) VALUES (21, 'Los Angeles', 'Sunset', '90001')
INTO Address (CustomerID, Town, Street, Zip) VALUES (31, 'Chicago', 'Michigan', '60601')
INTO Address (CustomerID, Town, Street, Zip) VALUES (41, 'Houston', 'MainSt', '77081')
INTO Address (CustomerID, Town, Street, Zip) VALUES (51, 'Phoenix', 'Central Ave', '85081')
INTO Address (CustomerID, Town, Street, Zip) VALUES (61, 'San Antonio', 'Alamo', '78201')
INTO Address (CustomerID, Town, Street, Zip) VALUES (71, 'San Diego', 'Broadway', '92101')
INTO Address (CustomerID, Town, Street, Zip) VALUES (81, 'Dallas', 'ElmSt', '75201')
INTO Address (CustomerID, Town, Street, Zip) VALUES (91, 'San Jose', 'SantaSt', '95181')
INTO Address (CustomerID, Town, Street, Zip) VALUES (100, 'Austin', 'Congress', '33301')
SELECT * FROM DUAL;
SELECT *FROM Address;
```

**Results** Explain Describe Saved SQL History

| CUSTOMERID | TOWN | STREET | ZIP |
|---|---|---|---|
| 11 | New York | Avenue | 10001 |
| 21 | Los Angeles | Sunset | 90001 |
| 31 | Chicago | Michigan | 60601 |
| 41 | Houston | MainSt | 77081 |
| 51 | Phoenix | Central Ave | 85081 |
| 61 | San Antonio | Alamo | 78201 |
| 71 | San Diego | Broadway | 92101 |
| 81 | Dallas | ElmSt | 75201 |
| 91 | San Jose | SantaSt | 95181 |
| 100 | Austin | Congress | 33301 |

10 rows returned in 0.01 seconds CSV Export

3.Orders:

```
INSERT ALL
INTO Orders (OrderID, OrderDate, Status, VAT, Discount) VALUES (12, '01-JAN-25', 'Pending', 5.00, 18.00)
INTO Orders (OrderID, OrderDate, Status, VAT, Discount) VALUES (22, '01-FEB-25', 'Shipped', 7.50, 5.00)
INTO Orders (OrderID, OrderDate, Status, VAT, Discount) VALUES (32, '01-MAR-25', 'Delivered', 8.00, 15.00)
INTO Orders (OrderID, OrderDate, Status, VAT, Discount) VALUES (42, '01-APR-25', 'Pending', 6.50, 8.00)
INTO Orders (OrderID, OrderDate, Status, VAT, Discount) VALUES (52, '01-MAY-25', 'Shipped', 10.00, 20.00)
INTO Orders (OrderID, OrderDate, Status, VAT, Discount) VALUES (62, '01-JUN-25', 'Delivered', 12.50, 25.00)
INTO Orders (OrderID, OrderDate, Status, VAT, Discount) VALUES (72, '01-JUL-25', 'Pending', 4.50, 18.00)
INTO Orders (OrderID, OrderDate, Status, VAT, Discount) VALUES (82, '01-AUG-25', 'Shipped', 9.00, 0.08)
INTO Orders (OrderID, OrderDate, Status, VAT, Discount) VALUES (92, '01-SEP-25', 'Delivered', 6.00, 5.00)
INTO Orders (OrderID, OrderDate, Status, VAT, Discount) VALUES (100,'01-OCT-25', 'Pending', 3.50, 30.00)
SELECT * FROM DUAL;
SELECT *FROM Orders;
```

**Results** Explain Describe Saved SQL History

| ORDERID | ORDERDATE | STATUS | VAT | DISCOUNT |
|---|---|---|---|---|
| 12 | 01-JAN-25 | Pending | 5 | 18 |
| 22 | 01-FEB-25 | Shipped | 7.5 | 5 |
| 32 | 01-MAR-25 | Delivered | 8 | 15 |
| 42 | 01-APR-25 | Pending | 6.5 | 8 |
| 52 | 01-MAY-25 | Shipped | 10 | 20 |
| 62 | 01-JUN-25 | Delivered | 12.5 | 25 |
| 72 | 01-JUL-25 | Pending | 4.5 | 18 |
| 82 | 01-AUG-25 | Shipped | 9 | .08 |
| 92 | 01-SEP-25 | Delivered | 6 | 5 |
| 100 | 01-OCT-25 | Pending | 3.5 | 30 |

10 rows returned in 0.01 seconds CSV Export

4.PhoneNumber:

```
INSERT ALL
INTO PhoneNumber (CustomerID, PhnNo) VALUES (11, '123-456-7890')
INTO PhoneNumber (CustomerID, PhnNo) VALUES (21, '2345678901')
INTO PhoneNumber (CustomerID, PhnNo) VALUES (31, '3456789012')
INTO PhoneNumber (CustomerID, PhnNo) VALUES (41, '4567890123')
INTO PhoneNumber (CustomerID, PhnNo) VALUES (51, '5678901234')
INTO PhoneNumber (CustomerID, PhnNo) VALUES (61, '6789012345')
INTO PhoneNumber (CustomerID, PhnNo) VALUES (71, '7890123456')
INTO PhoneNumber (CustomerID, PhnNo) VALUES (81, '8901234567')
INTO PhoneNumber (CustomerID, PhnNo) VALUES (91, '9012345678')
INTO PhoneNumber (CustomerID, PhnNo) VALUES (100, '10123456789')
SELECT *FROM DUAL;
SELECT *FROM PhoneNumber;
```

**Results** Explain Describe Saved SQL History

| CUSTOMERID | PHNNO |
|---|---|
| 11 | 123-456-7890 |
| 21 | 2345678901 |
| 31 | 3456789012 |
| 41 | 4567890123 |
| 51 | 5678901234 |
| 61 | 6789012345 |
| 71 | 7890123456 |
| 81 | 8901234567 |
| 91 | 9012345678 |
| 100 | 10123456789 |

10 rows returned in 0.00 seconds CSV Export

5.Customer_Order :

```
INSERT ALL
INTO CustomerOrder (CustomerID, OrderID) VALUES (11, 12)
INTO CustomerOrder (CustomerID, OrderID) VALUES (21, 22)
INTO CustomerOrder (CustomerID, OrderID) VALUES (31, 32)
INTO CustomerOrder (CustomerID, OrderID) VALUES (41, 42)
INTO CustomerOrder (CustomerID, OrderID) VALUES (51, 52)
INTO CustomerOrder (CustomerID, OrderID) VALUES (61, 62)
INTO CustomerOrder (CustomerID, OrderID) VALUES (71, 72)
INTO CustomerOrder (CustomerID, OrderID) VALUES (81, 82)
INTO CustomerOrder (CustomerID, OrderID) VALUES (91, 92)
INTO CustomerOrder (CustomerID, OrderID) VALUES (100, 100)
SELECT * FROM DUAL;
SELECT *FROM CustomerOrder;
```

**Results** Explain Describe Saved SQL History

| CUSTOMERID | ORDERID |
|---|---|
| 11 | 12 |
| 21 | 22 |
| 31 | 32 |
| 41 | 42 |
| 51 | 52 |
| 61 | 62 |
| 71 | 72 |
| 81 | 82 |
| 91 | 92 |
| 100 | 100 |

10 rows returned in 0.00 seconds CSV Export

6.Products

```
INSERT ALL
INTO Products (ProductID, ProductName, Price) VALUES (101, 'Laptop', 999.99)
INTO Products (ProductID, ProductName, Price) VALUES (201, 'Smartphone', 699.50)
INTO Products (ProductID, ProductName, Price) VALUES (301, 'Tablet', 499.00)
INTO Products (ProductID, ProductName, Price) VALUES (401, 'Headphones', 199.99)
INTO Products (ProductID, ProductName, Price) VALUES (501, 'Smartwatch', 299.49)
INTO Products (ProductID, ProductName, Price) VALUES (601, 'Gaming Console', 399.95)
INTO Products (ProductID, ProductName, Price) VALUES (701, 'Monitor', 249.99)
INTO Products (ProductID, ProductName, Price) VALUES (801, 'Keyboard', 49.99)
INTO Products (ProductID, ProductName, Price) VALUES (901, 'Mouse', 29.99)
INTO Products (ProductID, ProductName, Price) VALUES (1001, 'Printer', 149.99)
SELECT * FROM DUAL;
SELECT *FROM Products;
```

**Results** Explain Describe Saved SQL History

| PRODUCTID | PRODUCTNAME | PRICE |
|---|---|---|
| 101 | Laptop | 999.99 |
| 201 | Smartphone | 699.5 |
| 301 | Tablet | 499 |
| 401 | Headphones | 199.99 |
| 501 | Smartwatch | 299.49 |
| 601 | Gaming Console | 399.95 |
| 701 | Monitor | 249.99 |
| 801 | Keyboard | 49.99 |
| 901 | Mouse | 29.99 |
| 1001 | Printer | 149.99 |

10 rows returned in 0.00 seconds CSV Export

7.ProductDetails:

```
INSERT ALL
INTO ProductDetails (ProductID, Category, StockQuantity, Warranty, Brand) VALUES (101, 'Electronics', 50, '1 Year', 'Dell')
INTO ProductDetails (ProductID, Category, StockQuantity, Warranty, Brand) VALUES (201, 'Electronics', 100, '2 Years', 'Apple')
INTO ProductDetails (ProductID, Category, StockQuantity, Warranty, Brand) VALUES (301, 'Electronics', 75, '1 Year', 'Samsung')
INTO ProductDetails (ProductID, Category, StockQuantity, Warranty, Brand) VALUES (401, 'Accessories', 200, '6 Months', 'Sony')
INTO ProductDetails (ProductID, Category, StockQuantity, Warranty, Brand) VALUES (501, 'Wearables', 120, '1 Year', 'Fitbit')
INTO ProductDetails (ProductID, Category, StockQuantity, Warranty, Brand) VALUES (601, 'Gaming', 40, '3 Years', 'Microsoft')
INTO ProductDetails (ProductID, Category, StockQuantity, Warranty, Brand) VALUES (701, 'Peripherals', 88, '1 Year', 'LG')
INTO ProductDetails (ProductID, Category, StockQuantity, Warranty, Brand) VALUES (801, 'Peripherals', 150, '6 Months',
'Logitech')
INTO ProductDetails (ProductID, Category, StockQuantity, Warranty, Brand) VALUES (901, 'Peripherals', 300, '16 Months', 'HP')
INTO ProductDetails (ProductID, Category, StockQuantity, Warranty, Brand) VALUES (1001, 'Office Equipment', 60, '1
Year', 'Canon')
SELECT *FROM DUAL;
SELECT *FROM ProductDetails;
```

**Results** Explain Describe Saved SQL History

| PRODUCTID | CATEGORY | STOCKQUANTITY | WARRANTY | BRAND |
|---|---|---|---|---|
| 101 | Electronics | 50 | 1 Year | Dell |
| 201 | Electronics | 100 | 2 Years | Apple |
| 301 | Electronics | 75 | 1 Year | Samsung |
| 401 | Accessories | 200 | 6 Months | Sony |
| 501 | Wearables | 120 | 1 Year | Fitbit |
| 601 | Gaming | 40 | 3 Years | Microsoft |
| 701 | Peripherals | 88 | 1 Year | LG |
| 801 | Peripherals | 150 | 6 Months | Logitech |
| 901 | Peripherals | 300 | 16 Months | HP |
| 1001 | Office Equipment | 60 | 1 Year | Canon |

10 rows returned in 0.00 seconds CSV Export

8.OrderProducts :

```
INSERT ALL
INTO OrderProducts (OrderID, ProductID) VALUES (12, 101)
INTO OrderProducts (OrderID, ProductID) VALUES (22, 201)
INTO OrderProducts (OrderID, ProductID) VALUES (32, 301)
INTO OrderProducts (OrderID, ProductID) VALUES (42, 401)
INTO OrderProducts (OrderID, ProductID) VALUES (52, 501)
INTO OrderProducts (OrderID, ProductID) VALUES (62, 601)
INTO OrderProducts (OrderID, ProductID) VALUES (72, 701)
INTO OrderProducts (OrderID, ProductID) VALUES (82, 801)
INTO OrderProducts (OrderID, ProductID) VALUES (92, 901)
INTO OrderProducts (OrderID, ProductID) VALUES (100, 1001)
SELECT * FROM DUAL;
SELECT *FROM OrderProducts;
```

**Results** Explain Describe Saved SQL History

| ORDERID | PRODUCTID |
|---|---|
| 12 | 101 |
| 22 | 201 |
| 32 | 301 |
| 42 | 401 |
| 52 | 501 |
| 62 | 601 |
| 72 | 701 |
| 82 | 801 |
| 92 | 901 |
| 100 | 1001 |

10 rows returned in 0.00 seconds CSV Export

9.Payments:

```
INSERT ALL
INTO Payments (PaymentID, FinalAmount) VALUES (105, 999.99)
INTO Payments (PaymentID, FinalAmount) VALUES (205, 749.50)
INTO Payments (PaymentID, FinalAmount) VALUES (305, 1500.00)
INTO Payments (PaymentID, FinalAmount) VALUES (405, 300.75)
INTO Payments (PaymentID, FinalAmount) VALUES (505, 1200.25)
INTO Payments (PaymentID, FinalAmount) VALUES (605, 499.99)
INTO Payments (PaymentID, FinalAmount) VALUES (705, 250.00)
INTO Payments (PaymentID, FinalAmount) VALUES (805, 1800.40)
INTO Payments (PaymentID, FinalAmount) VALUES (905, 350.60)
INTO Payments (PaymentID, FinalAmount) VALUES (1005, 1000.00)
SELECT * FROM DUAL;
SELECT *FROM Payments;
```

Results Explain Describe Saved SQL History

| PAYMENTID | FINALAMOUNT |
|---|---|
| 105 | 999.99 |
| 205 | 749.5 |
| 305 | 1500 |
| 405 | 300.75 |
| 505 | 1200.25 |
| 605 | 499.99 |
| 705 | 250 |
| 805 | 1800.4 |
| 905 | 350.6 |
| 1005 | 1000 |

10 rows returned in 0.01 seconds CSV Export

10.Payment_Method:

```
INSERT ALL
INTO PaymentMethod (PaymentID, PaymentMethod) VALUES (105, 'Credit Card')
INTO PaymentMethod (PaymentID, PaymentMethod) VALUES (205, 'Debit Card')
INTO PaymentMethod (PaymentID, PaymentMethod) VALUES (305, 'PayPal')
INTO PaymentMethod (PaymentID, PaymentMethod) VALUES (405, 'Bank Transfer')
INTO PaymentMethod (PaymentID, PaymentMethod) VALUES (505, 'Cash')
INTO PaymentMethod (PaymentID, PaymentMethod) VALUES (605, 'Google Pay')
INTO PaymentMethod (PaymentID, PaymentMethod) VALUES (705, 'Apple Pay')
INTO PaymentMethod (PaymentID, PaymentMethod) VALUES (805, 'Cryptocurrency')
INTO PaymentMethod (PaymentID, PaymentMethod) VALUES (905, 'Amazon Pay')
INTO PaymentMethod (PaymentID, PaymentMethod) VALUES (1005, 'Check')
SELECT *FROM DUAL;
SELECT *FROM PaymentMethod;
```

**Results** **Explain** **Describe** **Saved SQL** **History**

| PAYMENTID | PAYMENTMETHOD |
|---|---|
| 105 | Credit Card |
| 205 | Debit Card |
| 305 | PayPal |
| 405 | Bank Transfer |
| 505 | Cash |
| 605 | Google Pay |
| 705 | Apple Pay |
| 805 | Cryptocurrency |
| 905 | Amazon Pay |
| 1005 | Check |

10 rows returned in 0.02 seconds CSV Export

11.Order_Payment :

```
INSERT ALL
INTO OrderPayment (OrderID, PaymentID) VALUES (12, 105)
INTO OrderPayment (OrderID, PaymentID) VALUES (22, 205)
INTO OrderPayment (OrderID, PaymentID) VALUES (32, 305)
INTO OrderPayment (OrderID, PaymentID) VALUES (42, 405)
INTO OrderPayment (OrderID, PaymentID) VALUES (52, 505)
INTO OrderPayment (OrderID, PaymentID) VALUES (62, 605)
INTO OrderPayment (OrderID, PaymentID) VALUES (72, 705)
INTO OrderPayment (OrderID, PaymentID) VALUES (82, 805)
INTO OrderPayment (OrderID, PaymentID) VALUES (92, 905)
INTO OrderPayment (OrderID, PaymentID) VALUES (100, 1005)
SELECT * FROM DUAL;
SELECT *FROM OrderPayment;
```

**Results** **Explain** **Describe** **Saved SQL** **History**

| ORDERID | PAYMENTID |
|---|---|
| 12 | 105 |
| 22 | 205 |
| 32 | 305 |
| 42 | 405 |
| 52 | 505 |
| 62 | 605 |
| 72 | 705 |
| 82 | 805 |
| 92 | 905 |
| 100 | 1005 |

10 rows returned in 0.00 seconds CSV Export

12.Employees:

```
INSERT ALL
INTO Employees (EmployeeID, EmployeeName, DOB) VALUES (103, 'Alice', '15-SEP-1990')
INTO Employees (EmployeeID, EmployeeName, DOB) VALUES (203, 'Smith', '2-FEB-1985')
INTO Employees (EmployeeID, EmployeeName, DOB) VALUES (303, 'Brown', '7-MAY-1992')
INTO Employees (EmployeeID, EmployeeName, DOB) VALUES (403, 'David', '21-MAR-1980')
INTO Employees (EmployeeID, EmployeeName, DOB) VALUES (503, 'Eveli', '12-JAN-1995')
INTO Employees (EmployeeID, EmployeeName, DOB) VALUES (603, 'Frankin', '9-SEP-1988')
INTO Employees (EmployeeID, EmployeeName, DOB) VALUES (703, 'Leo', '13-OCT-1991')
INTO Employees (EmployeeID, EmployeeName, DOB) VALUES (803, 'Wilson', '11-DEC-1983')
INTO Employees (EmployeeID, EmployeeName, DOB) VALUES (903, 'Scott', '23-JUL-1994')
INTO Employees (EmployeeID, EmployeeName, DOB) VALUES (1003, 'Taylor', '17-NOV-1999')
SELECT * FROM DUAL;
SELECT *FROM Employees;
```

**Results** Explain Describe Saved SQL History

| EMPLOYEEID | EMPLOYEENAME | DOB |
|---|---|---|
| 103 | Alice | 15-SEP-90 |
| 203 | Smith | 02-FEB-85 |
| 303 | Brown | 07-MAY-92 |
| 403 | David | 21-MAR-80 |
| 503 | Eveli | 12-JAN-95 |
| 603 | Frankin | 09-SEP-88 |
| 703 | Leo | 13-OCT-91 |
| 803 | Wilson | 11-DEC-83 |
| 903 | Scott | 23-JUL-94 |
| 1003 | Taylor | 17-NOV-99 |

10 rows returned in 0.00 seconds CSV Export

13.Employee_Details:

```
INSERT ALL
INTO EmployeeDetails (EmployeeID, Post, Comm, Email) VALUES (103, 'Manager', 1500.00, 'alice.johnson@email.com')
INTO EmployeeDetails (EmployeeID, Post, Comm, Email) VALUES (203, 'Developer', 1200.50, 'bob.smith@email.com')
INTO EmployeeDetails (EmployeeID, Post, Comm, Email) VALUES (303, 'HR', 900.00, 'charlie.brown@email.com')
INTO EmployeeDetails (EmployeeID, Post, Comm, Email) VALUES (403, 'Sales', 1100.75, 'david.clark@email.com')
INTO EmployeeDetails (EmployeeID, Post, Comm, Email) VALUES (503, 'Marketing', 1003.08, 'eve.white@email.com')
INTO EmployeeDetails (EmployeeID, Post, Comm, Email) VALUES (603, 'Designer', 950.25, 'frank.harris@email.com')
INTO EmployeeDetails (EmployeeID, Post, Comm, Email) VALUES (703, 'Engineer', 1350.00, 'grace.lee@email.com')
INTO EmployeeDetails (EmployeeID, Post, Comm, Email) VALUES (803, 'Accountant', 1100.50, 'hannah.wilson@email.com')
INTO EmployeeDetails (EmployeeID, Post, Comm, Email) VALUES (903, 'Consultant', 1400.75, 'ivy.scott@email.com')
INTO EmployeeDetails (EmployeeID, Post, Comm, Email) VALUES (1003, 'CEO', 2000.08, 'jack.taylor@email.com')
SELECT * FROM DUAL;
SELECT * FROM EmployeeDetails;
```

Results Explain Describe Saved SQL History

| EMPLOYEEID | POST | COMM | EMAIL |
|---|---|---|---|
| 103 | Manager | 1500 | alice.johnson@email.com |
| 203 | Developer | 1200.5 | bob.smith@email.com |
| 303 | HR | 900 | charlie.brown@email.com |
| 403 | Sales | 1100.75 | david.clark@email.com |
| 503 | Marketing | 1003.08 | eve.white@email.com |
| 603 | Designer | 950.25 | frank.harris@email.com |
| 703 | Engineer | 1350 | grace.lee@email.com |
| 803 | Accountant | 1100.5 | hannah.wilson@email.com |
| 903 | Consultant | 1400.75 | ivy.scott@email.com |
| 1003 | CEO | 2000.08 | jack.taylor@email.com |

10 rows returned in 0.02 seconds CSV Export

14.EmployeePhones:

```
INSERT ALL
INTO EmployeePhones (EmployeeID, PhnNo) VALUES (103, '123-456-7890')
INTO EmployeePhones (EmployeeID, PhnNo) VALUES (203, '234-567-8901')
INTO EmployeePhones (EmployeeID, PhnNo) VALUES (303, '345-678-9012')
INTO EmployeePhones (EmployeeID, PhnNo) VALUES (403, '456-789-0123')
INTO EmployeePhones (EmployeeID, PhnNo) VALUES (503, '567-890-1234')
INTO EmployeePhones (EmployeeID, PhnNo) VALUES (603, '678-901-2345')
INTO EmployeePhones (EmployeeID, PhnNo) VALUES (703, '789-012-3456')
INTO EmployeePhones (EmployeeID, PhnNo) VALUES (803, '890-123-4567')
INTO EmployeePhones (EmployeeID, PhnNo) VALUES (903, '901-234-5678')
INTO EmployeePhones (EmployeeID, PhnNo) VALUES (1003, '012-345-6789')
SELECT * FROM DUAL;
SELECT *FROM EmployeePhones;
```

**Results** **Explain** **Describe** **Saved SQL** **History**

| EMPLOYEEID | PHNNO |
|---|---|
| 103 | 123-456-7890 |
| 203 | 234-567-8901 |
| 303 | 345-678-9012 |
| 403 | 456-789-0123 |
| 503 | 567-890-1234 |
| 603 | 678-901-2345 |
| 703 | 789-012-3456 |
| 803 | 890-123-4567 |
| 903 | 901-234-5678 |
| 1003 | 012-345-6789 |

10 rows returned in 0.00 seconds CSV Export

15.EmployeeOrders:

```
INSERT ALL
INTO EmployeeOrders (OrderID, EmployeeID) VALUES (12, 103)
INTO EmployeeOrders (OrderID, EmployeeID) VALUES (22, 203)
INTO EmployeeOrders (OrderID, EmployeeID) VALUES (32, 303)
INTO EmployeeOrders (OrderID, EmployeeID) VALUES (42, 403)
INTO EmployeeOrders (OrderID, EmployeeID) VALUES (52, 503)
INTO EmployeeOrders (OrderID, EmployeeID) VALUES (62, 603)
INTO EmployeeOrders (OrderID, EmployeeID) VALUES (72, 703)
INTO EmployeeOrders (OrderID, EmployeeID) VALUES (82, 803)
INTO EmployeeOrders (OrderID, EmployeeID) VALUES (92, 903)
INTO EmployeeOrders (OrderID, EmployeeID) VALUES (100, 1003)
SELECT * FROM DUAL;
SELECT *FROM EmployeeOrders;
```

Results Explain Describe Saved SQL History

| ORDERID | EMPLOYEEID |
|---|---|
| 12 | 103 |
| 22 | 203 |
| 32 | 303 |
| 42 | 403 |
| 52 | 503 |
| 62 | 603 |
| 72 | 703 |
| 82 | 803 |
| 92 | 903 |
| 100 | 1003 |

10 rows returned in 0.00 seconds CSV Export

16.Suppliers:

```
INSERT ALL
INTO Suppliers (SupplierID, SupplierName, PaymentDetails, SupplyPrice, SupplyDate) VALUES (107, 'ABC
Electronics', 'Bank Transfer', 150.75, '17-JAN-2005')
INTO Suppliers (SupplierID, SupplierName, PaymentDetails, SupplyPrice, SupplyDate) VALUES (287, '2017
Gadgets', 'Cash on Delivery', 95.50, '01-FEB-2025')
INTO Suppliers (SupplierID, SupplierName, PaymentDetails, SupplyPrice, SupplyDate) VALUES (387, 'Tech
Supplies Inc.', 'Credit Card', 250.00, '13-MAR-2025')
INTO Suppliers (SupplierID, SupplierName, PaymentDetails, SupplyPrice, SupplyDate) VALUES (487, 'Quality
Products Ltd.', 'Cash on Delivery', 120.00, '29-JAN-2025')
INTO Suppliers (SupplierID, SupplierName, PaymentDetails, SupplyPrice, SupplyDate) VALUES (587, 'Modern
Tech Supplies', 'Bank Transfer', 80.20, '11-MAR-2025')
INTO Suppliers (SupplierID, SupplierName, PaymentDetails, SupplyPrice, SupplyDate) VALUES (607, 'Global
Trading Co.', 'Bank Transfer', 300.00, '19-FEB-2025')
INTO Suppliers (SupplierID, SupplierName, PaymentDetails, SupplyPrice, SupplyDate) VALUES (787, 'Innovative
Electronics', 'Cheque', 145.90, '17-JAN-2025')
INTO Suppliers (SupplierID, SupplierName, PaymentDetails, SupplyPrice, SupplyDate) VALUES (887, 'Smart
Devices Ltd.', 'Cheque', 110.00, '12-APR-2025')
INTO Suppliers (SupplierID, SupplierName, PaymentDetails, SupplyPrice, SupplyDate) VALUES (907, 'Best
Gadgets Inc.', 'Credit Card', 220.00, '21-JAN-2025')
INTO Suppliers (SupplierID, SupplierName, PaymentDetails, SupplyPrice, SupplyDate) VALUES (1007,
'ElectroHub', 'Cash on Delivery', 75.50, '19-FEB-2025')
 SELECT * FROM DUAL;
 SELECT *FROM Suppliers;
```

**Results** Explain Describe Saved SQL History

| SUPPLIERID | SUPPLIERNAME | PAYMENTDETAILS | SUPPLYPRICE | SUPPLYDATE |
|---|---|---|---|---|
| 107 | ABC Electronics | Bank Transfer | 150.75 | 17-JAN-05 |
| 287 | 2017 Gadgets | Cash on Delivery | 95.5 | 01-FEB-25 |
| 387 | Tech Supplies Inc. | Credit Card | 250 | 13-MAR-25 |
| 487 | Quality Products Ltd. | Cash on Delivery | 120 | 29-JAN-25 |
| 587 | Modern Tech Supplies | Bank Transfer | 80.2 | 11-MAR-25 |
| 607 | Global Trading Co. | Bank Transfer | 300 | 19-FEB-25 |
| 787 | Innovative Electronics | Cheque | 145.9 | 17-JAN-25 |
| 887 | Smart Devices Ltd. | Cheque | 110 | 12-APR-25 |
| 907 | Best Gadgets Inc. | Credit Card | 220 | 21-JAN-25 |
| 1007 | ElectroHub | Cash on Delivery | 75.5 | 19-FEB-25 |

10 rows returned in 0.00 seconds CSV Export

17.SuppliersProducts:

```
INSERT ALL
INTO SupplierProducts (SupplierID, ProductID) VALUES (107, 101)
INTO SupplierProducts (SupplierID, ProductID) VALUES (287, 201)
INTO SupplierProducts (SupplierID, ProductID) VALUES (387, 301)
INTO SupplierProducts (SupplierID, ProductID) VALUES (487, 401)
INTO SupplierProducts (SupplierID, ProductID) VALUES (587, 501)
INTO SupplierProducts (SupplierID, ProductID) VALUES (607, 601)
INTO SupplierProducts (SupplierID, ProductID) VALUES (787, 701)
INTO SupplierProducts (SupplierID, ProductID) VALUES (887, 801)
INTO SupplierProducts (SupplierID, ProductID) VALUES (907, 901)
INTO SupplierProducts (SupplierID, ProductID) VALUES (1007, 1001)
SELECT * FROM DUAL;
SELECT *FROM SupplierProducts;
```

Results Explain Describe Saved SQL History

| SUPPLIERID | PRODUCTID |
|---|---|
| 107 | 101 |
| 287 | 201 |
| 387 | 301 |
| 487 | 401 |
| 587 | 501 |
| 607 | 601 |
| 787 | 701 |
| 887 | 801 |
| 907 | 901 |
| 1007 | 1001 |

10 rows returned in 0.02 seconds CSV Export

# Query Test in DB

## Simple Query:

1. Write a query to display the employee_Id, post, Email of all employees who earn a Commission.

```
SELECT EmployeeID, Post, Email
FROM EmployeeDetails
WHERE Comm IS NOT NULL;
```

Results Explain Describe Saved SQL History

| EMPLOYEEID | POST | EMAIL |
|---|---|---|
| 103 | Manager | alice.johnson@email.com |
| 203 | Developer | bob.smith@email.com |
| 303 | HR | charlie.brown@email.com |
| 403 | Sales | david.clark@email.com |
| 503 | Marketing | eve.white@email.com |
| 603 | Designer | frank.harris@email.com |
| 703 | Engineer | grace.lee@email.com |
| 803 | Accountant | hannah.wilson@email.com |
| 903 | Consultant | ivy.scott@email.com |
| 1003 | CEO | jack.taylor@email.com |

10 rows returned in 0.01 seconds CSV Export

2. Write a query to display the Product_ID, Category, and Brand of products that come with a 1-year warranty.

```
SELECT ProductID, Category, Brand
FROM ProductDetails
WHERE Warranty = '1 Year';
```

Results Explain Describe Saved SQL History

| PRODUCTID | CATEGORY | BRAND |
|---|---|---|
| 101 | Electronics | Dell |
| 301 | Electronics | Samsung |
| 501 | Wearables | Fitbit |
| 701 | Peripherals | LG |
| 1001 | Office Equipment | Canon |

5 rows returned in 0.00 seconds CSV Export

## Single row function Query:

1.Write a query that will display the supplier's name with the first letter capitalized and all other letter lowercase, for all employees whose name starts with S, G, or B. Also, suppliers for fewer than 100 months, display the supply date, three-month review date, first Sunday after hire date when hired. Format the dates to appear in the format similar to "13 September, 1981."

```
       SELECT SupplierID,
       INITCAP(SupplierName) AS SupplierName,
       TO_CHAR(SupplyDate, 'DD Month, YYYY') AS SupplyDate,
       TO_CHAR(ADD_MONTHS(SupplyDate, 3), 'DD Month, YYYY') AS Three_Month_Review_Date,
       TO_CHAR(NEXT_DAY(SupplyDate,'SUNDAY'), 'DD Month, YYYY') AS First_Sunday_After_Supply
       FROM Suppliers
       WHERE (SupplierName LIKE 'S%' OR SupplierName LIKE 'G%' OR SupplierName LIKE 'B%') AND
MONTHS_BETWEEN(SYSDATE, SupplyDate) < 100;
```

Results Explain Describe Saved SQL History

| SUPPLIERID | SUPPLIERNAME | SUPPLYDATE | THREE_MONTH_REVIEW_DATE | FIRST_SUNDAY_AFTER_SUPPLY |
|---|---|---|---|---|
| 607 | Global Trading Co. | 19 February , 2025 | 19 May , 2025 | 23 February , 2025 |
| 887 | Smart Devices Ltd. | 12 April , 2025 | 12 July , 2025 | 13 April , 2025 |
| 907 | Best Gadgets Inc. | 21 January , 2025 | 21 April , 2025 | 26 January , 2025 |

**Multiple row function/ aggregate function Query:**

1. Write a query to display the total stock quantity, average stock quantity, and count of products for each category in the product details. Include only products with a 1-year warranty and filter the categories to include only "Electronics" and "Peripherals".

```
SELECT Category,
    SUM(StockQuantity) AS Total_Stock_Quantity,
    AVG(StockQuantity) AS Average_Stock_Quantity
    FROM ProductDetails
    WHERE
    Warranty = '1 Year'
    AND Category IN ('Electronics', 'Peripherals')
    GROUP BY Category;
```

Results Explain Describe Saved SQL History

| CATEGORY | TOTAL_STOCK_QUANTITY | AVERAGE_STOCK_QUANTITY |
|---|---|---|
| Electronics | 125 | 62.5 |
| Peripherals | 88 | 88 |

2 rows returned in 0.01 seconds CSV Export

## Single row Sub-Query:

1.Display the supplier name, supplier ID, and payment details of the supplier associated with the product that has the highest stock quantity.

```
SELECT SupplierName, SupplierID, PaymentDetails
FROM Suppliers
WHERE SupplierID = (
    SELECT SupplierID
    FROM SupplierProducts
    WHERE ProductID = (
        SELECT ProductID
        FROM ProductDetails
        WHERE StockQuantity = (SELECT MAX(StockQuantity) FROM ProductDetails)
    ));
```

Results Explain Describe Saved SQL History

| SUPPLIERNAME | SUPPLIERID | PAYMENTDETAILS |
|---|---|---|
| Best Gadgets Inc. | 907 | Credit Card |

1 rows returned in 0.02 seconds CSV Export

2.Display the name of the employee who processed the order with the earliest order date.

```
SELECT EmployeeName
FROM Employees
WHERE EmployeeID = (
    SELECT EmployeeID
    FROM EmployeeOrders
    WHERE OrderID = (
        SELECT OrderID
        FROM Orders
        WHERE OrderDate = (SELECT MIN(OrderDate) FROM Orders)));
```

Results Explain Describe Saved SQL History

| EMPLOYEENAME |
|---|
| Alice |

1 rows returned in 0.01 seconds CSV Export

## Multiple row Sub-Query:

1.Display all customers who have placed orders with a status of "Shipped" or "Delivered".

```
SELECT CustomerID, CustomerName
FROM Customer
WHERE CustomerID IN (
    SELECT CustomerID
    FROM Orders
    WHERE Status IN ('Shipped', 'Delivered'));
```

Results Explain Describe Saved SQL History

| CUSTOMERID | CUSTOMERNAME |
|---|---|
| 11 | John |
| 21 | Jane |
| 31 | Alice |
| 41 | Bob |
| 51 | Charlie |
| 61 | Diana |
| 71 | Edward |
| 81 | Fiona |
| 91 | George |
| 100 | Helen |

10 rows returned in 0.00 seconds CSV Export

2.Display the Order_IDs, VAT,Discount for all orders that have a payment amount greater than the average payment amount.

```
SELECT OrderID, VAT, Discount
FROM Orders
WHERE OrderID IN (
    SELECT OrderID
    FROM Payments
    WHERE FinalAmount > (SELECT AVG(FinalAmount) FROM Payments));
```

Results Explain Describe Saved SQL History

| ORDERID | VAT | DISCOUNT |
|---|---|---|
| 12 | 5 | 18 |
| 22 | 7.5 | 5 |
| 32 | 8 | 15 |
| 42 | 6.5 | 8 |
| 52 | 10 | 20 |
| 62 | 12.5 | 25 |
| 72 | 4.5 | 18 |
| 82 | 9 | .08 |
| 92 | 6 | 5 |
| 100 | 3.5 | 30 |

10 rows returned in 0.00 seconds CSV Export

## Joining Query:

1. Display the order ID, product ID, and product name for all orders that contain the product "Laptop". (**Using Equijoin**)

```
SELECT op.OrderID, op.ProductID, p.ProductName
FROM Orders o, Products p, OrderProducts op
WHERE op.OrderID = o.OrderID
  AND p.ProductName = 'Laptop';
```

Results Explain Describe Saved SQL History

| ORDERID | VAT | DISCOUNT |
|---|---|---|
| 12 | 5 | 18 |
| 22 | 7.5 | 5 |
| 32 | 8 | 15 |
| 42 | 6.5 | 8 |
| 52 | 10 | 20 |
| 62 | 12.5 | 25 |
| 72 | 4.5 | 18 |
| 82 | 9 | .08 |
| 92 | 6 | 5 |
| 100 | 3.5 | 30 |

10 rows returned in 0.00 seconds CSV Export

2. Display all the products along with their supplier names, and if a product does not have a supplier, display it as ‘No Supplier Available’ and only include suppliers who have 'Credit Card' as their payment method. (**Using Outer join**)

```
SELECT  p.ProductName, NVL(s.SupplierName, 'No Supplier Available') AS SupplierName
FROM
Products p, Suppliers s, SupplierProducts sp
WHERE
 sp.SupplierId=s.SupplierId(+)
 AND s.PaymentDetails='Credit Card';
```

Results Explain Describe Saved SQL History

| PRODUCTNAME | SUPPLIERNAME |
|---|---|
| Laptop | Tech Supplies Inc. |
| Smartphone | Tech Supplies Inc. |
| Tablet | Tech Supplies Inc. |
| Headphones | Tech Supplies Inc. |
| Smartwatch | Tech Supplies Inc. |
| Gaming Console | Tech Supplies Inc. |
| Monitor | Tech Supplies Inc. |
| Keyboard | Tech Supplies Inc. |
| Mouse | Tech Supplies Inc. |
| Printer | Tech Supplies Inc. |

3.List all the employees who are earning commission more than $1200. (**Using Self join**).

```
SELECT e1.EmployeeID, e2.EmployeeName,e1.comm
FROM EmployeeDetails e1, Employees e2
WHERE
     e1.EmployeeID = e2.EmployeeID
     AND e1.comm > 1200;
```

**Results** Explain Describe Saved SQL History

| EMPLOYEEID | EMPLOYEENAME | COMM |
|---|---|---|
| 103 | Alice | 1500 |
| 203 | Smith | 1200.5 |
| 703 | Leo | 1350 |
| 903 | Scott | 1400.75 |
| 1003 | Taylor | 2000.08 |

5 rows returned in 0.02 seconds CSV Export

4.List all products along with supplier names where the product price is greater than the supplier's minimum supply price. (Non-Equijoin)

```
SELECT p.product_id, p.product_name, p.price, s.supplier_name,
       s.supply_price
FROM
    Products p ,Supplierr s
WHERE
     p.price <= s.supply_price;
```

| PRODUCT_ID | PRODUCT_NAME | PRICE | SUPPLIER_NAME | SUPPLY_PRICE |
|---|---|---|---|---|
| 901 | Mouse | 29.99 | Smart Devices Ltd | 75.5 |
| 901 | Mouse | 29.99 | Best Gadgets Inc | 80.2 |
| 901 | Mouse | 29.99 | XYZ Gadgets | 95.5 |
| 901 | Mouse | 29.99 | Modern Tech Suppliers | 110 |
| 901 | Mouse | 29.99 | Global Trading Co. | 120 |
| 901 | Mouse | 29.99 | Innovative Electics | 145.9 |
| 901 | Mouse | 29.99 | ABC Electronics | 150.75 |
| 901 | Mouse | 29.99 | Tech Suppliers Inc | 250 |
| 901 | Mouse | 29.99 | ElectroHub | 270.3 |
| 901 | Mouse | 29.99 | Quality Products Ltd | 300 |
| More than 10 rows available. Increase rows selector to view more rows. | | | | |

**VIEW:**

1.Create a view called SUPPLIER_VU based on the supplier_id, supplier_name from the Suppliers table for suppliers who use 'Bank Transfer' as their payment method?

```
CREATE VIEW Supplier_VU
AS SELECT SupplierId, SupplierName
FROM Suppliers
WHERE PaymentDetails='Bank Transfer';
DESC Supplier_VU;
```

Results Explain Describe Saved SQL History

Object Type **VIEW** Object **SUPPLIER_VU**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| SUPPLIER_VU | SUPPLIERID | Number | - | - | - | - | - | - | - |
| | SUPPLIERNAME | Varchar2 | 255 | - | - | - | - | - | - |

1 - 2

Results Explain Describe Saved SQL History

| SUPPLIERID | SUPPLIERNAME |
|---|---|
| 107 | ABC Electronics |
| 587 | Modern Tech Supplies |
| 607 | Global Trading Co. |

3 rows returned in 0.00 seconds CSV Export

(1) Create a view named PRODUCT_VU that contains the product_id, stock_quantity, and brand for all product_details table in category Electronics & Peripherals. Label the view column PRODUCT_ID, STOCK_QUANTITY, and PRODUCT_BRAND. Do not allow a product to be reassigned to another category through the view.

```
CREATE VIEW PRODUCT_VU (ProductId, StockQuantity, ProductBrand)
AS SELECT ProductId, StockQuantity, Brand
FROM ProductDetails
WHERE Category IN ('Electronics', 'Peripherals')
WITH CHECK OPTION CONSTRAINT PRODUCT_VU_EP;
DESC PRODUCT_VU;
```

Results Explain Describe Saved SQL History

Object Type **VIEW** Object **PRODUCT_VU**

| Table | Column | Data Type | Length | Precision | Scale | Primary Key | Nullable | Default | Comment |
|---|---|---|---|---|---|---|---|---|---|
| PRODUCT_VU | PRODUCTID | Number | - | - | 0 | - | - | - | - |
| | STOCKQUANTITY | Number | - | - | 0 | - | ✓ | - | - |
| | PRODUCTBRAND | Varchar2 | 255 | - | - | - | ✓ | - | - |

1 - 3

Results Explain Describe Saved SQL History

| PRODUCTID | STOCKQUANTITY | PRODUCTBRAND |
|---|---|---|
| 101 | 50 | Dell |
| 201 | 100 | Apple |
| 301 | 75 | Samsung |
| 701 | 88 | LG |
| 801 | 150 | Logitech |
| 901 | 300 | HP |

6 rows returned in 0.00 seconds CSV Export

# Description of a Successful DB connection

## Setting Up Java-MySQL Connection

- **Getting the Connector/J:**

  To connect Java with MySQL, I got the necessary Connector/J JAR file from MySQL's official website.
  I went to "https://dev.mysql.com/downloads/connector/j/" and picked the right version for my project.

- **Installing MySQL Shell:**

  I made sure everything started well by downloading MySQL, Shell from the MySQL website.
  By following the installation instructions for my system, I made sure MySQL Shell was properly set up.

- **Creating Database and Tables:**

  In the MySQL Shell, I smartly set up the database structure using short but effective SQL commands.
  To make things easier, I kept a script with the DDL statements for reference.

- **Setting Up in IntelliJ IDEA:**

  I made things smooth in IntelliJ IDEA by organizing my project's parts neatly.

  Using the "Project Structure" settings, I nicely combined my Java code, resources, and outside stuff.

- **Strong Connection with Connector/J:**

  To make a solid connection, I added a new library to my project.
  I picked the right Connector/J JAR file that's good at connecting Java and MySQL.

## Setting Up MySQL Connectivity

- **Registering the Driver:**
  I initiated by registering the MySQL. JDBC driver. This step allows my Java code to understand how to communicate with the MySQL database.

- **Connecting to the Database:**
  Next, I established a connection to the MySQL database. This connection forms a secure pathway for my Java code to interact with the data.

- **Creating a Statement:**
  With the connection in place, I moved on to create a statement object. This object is like a messenger that conveys my SQL queries to the database.

- **Executing Queries and Obtaining Results:**
  I effectively executed my SQL queries using the statement object. This step resulted in a ResultSet, containing the data I needed from the database.

- **Closing the Connection:**
  After finishing the necessary operations, I ensured proper resource management by closing the connection. This step prevents unnecessary resource usage.

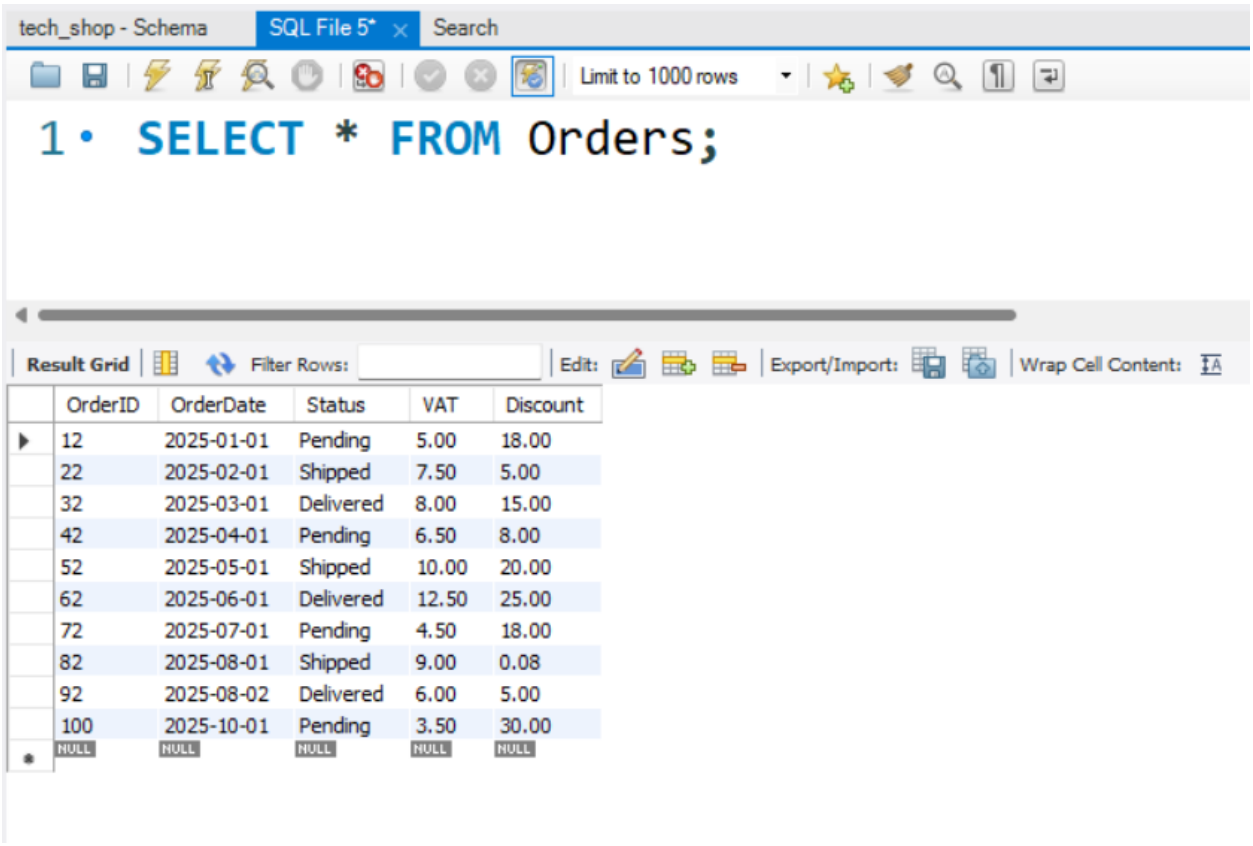



**Fig 6: Salary table**

MySQLConnect.java

```java
package com.example.project.classes;
import java.sql.*;
public class MySQLConnect {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/tech_shop?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true";
        String user = "project";
        String password = "techshop";
        try {
            Class.forName( className: "com.mysql.cj.jdbc.Driver");
            Connection conn = DriverManager.getConnection(url, user, password);
            if (conn != null) {
                System.out.println("Connected to the database successfully!");
                Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery( sql: "SELECT * FROM Orders");
                while (rs.next()) {
                    int orderId = rs.getInt( columnLabel: "OrderID");
                    Date orderDate = rs.getDate( columnLabel: "OrderDate");
                    String status = rs.getString( columnLabel: "Status");
                    double vat = rs.getDouble( columnLabel: "VAT");
                    double discount = rs.getDouble( columnLabel: "Discount");
                    System.out.println("Order ID: " + orderId + ", Order Date: "
                            + orderDate + ", Status: " + status + ", VAT: " + vat + ", Discount: " + discount);
                }rs.close();
                stmt.close();
            } else {System.out.println("Connection not established");}
            conn.close();
        } catch (ClassNotFoundException | SQLException e) {
            System.out.println("Error: " + e);
```

Run MySQLConnect

```
Order ID: 12, Order Date: 2025-01-01, Status: Pending, VAT: 5.0, Discount: 18.0
Order ID: 22, Order Date: 2025-02-01, Status: Shipped, VAT: 7.5, Discount: 5.0
Order ID: 32, Order Date: 2025-03-01, Status: Delivered, VAT: 8.0, Discount: 15.0
Order ID: 42, Order Date: 2025-04-01, Status: Pending, VAT: 6.5, Discount: 8.0
Order ID: 52, Order Date: 2025-05-01, Status: Shipped, VAT: 10.0, Discount: 20.0
Order ID: 62, Order Date: 2025-06-01, Status: Delivered, VAT: 12.5, Discount: 25.0
Order ID: 72, Order Date: 2025-07-01, Status: Pending, VAT: 4.5, Discount: 18.0
Order ID: 82, Order Date: 2025-08-01, Status: Shipped, VAT: 9.0, Discount: 0.08
Order ID: 92, Order Date: 2025-08-02, Status: Delivered, VAT: 6.0, Discount: 5.0
Order ID: 100, Order Date: 2025-10-01, Status: Pending, VAT: 3.5, Discount: 30.0

Process finished with exit code 0
```

**Fig 7: Database Connection**

## Features:

1. **Connection Pooling:** Efficiently manages database connections, reducing overhead and enhancing performance.

2. **ConfigLoader Utility:** Simplifies configuration management and enhances code readability, adaptable for both MySQL and Oracle SQL, and can be changed at any moment with ease,

allowing seamless integration with various database systems.

3. **Robust DAO Layer:** Provides structured data access with dedicated interfaces and implementations, ensuring clear separation of concerns.

4. **Encapsulated Query Constants:** Ensures secure and structured database queries, minimizing vulnerabilities and promoting code quality.

5. **MVC Design Pattern:** Organizes application components into Models, Views, and Controllers, fostering maintainability and scalability.

6. **Multi-Threading and Graceful Shutdown:** Enables a responsive user experience with background database connections, enhancing application performance.

# Conclusion

The Tech Shop Management System project has successfully developed a robust software tool that aids in streamlining the operations of a tech shop, enhancing efficiency, and improving customer service. Throughout this project, our team employed Oracle 10g for effective database management and Java for a user-friendly application interface, addressing the real-world challenges of managing sales, customer data, inventory, and financial records in a tech shop environment.

As we move forward, the future aspects of this project include plans for integrating more advanced analytics for predictive sales trends, incorporating AI to provide personalized shopping experiences, and expanding the system's compatibility with various mobile platforms to increase accessibility. Additionally, considering security enhancements and cloud-based solutions will be crucial to ensure data integrity and scalability. This project not only delivered a practical solution but also laid a solid foundation for future innovations that could transform tech shop management practices further.